

Building a QuantLib Derivatives Workbench

Human-directed engineering with Codex and Claude, from vanilla pricing to a callable cross-currency hybrid engine

Tim Launer with Codex and Claude – QuantLib, Python, C++, macOS,
and DigitalOcean Evidence through 22 August 2026

01 / Finance foundations

A derivative pricer is a disciplined cash-flow machine

A price is a conditional present value

- ▶ A derivative is a rule that maps future market states into dated cash flows.
- ▶ Pricing chooses a probability measure and discounts those cash flows with the associated numeraire.
- ▶ Implementation must preserve dates, currencies, signs, conventions, and information available at each decision time.

Takeaway. The model is not the product; it is the machinery used to value the product's cash-flow rules.

Four layers keep pricing logic intelligible

- ▶ Product layer: contractual schedules, coupons, notionals, exercise rights, and settlement rules.
- ▶ Market layer: curves, volatilities, FX spot/forwards, correlations, fixings, and quote conventions.
- ▶ Model and numerical layers: dynamics, calibration, solver, grid, simulation, regression, and tolerances.

Takeaway. Most serious pricing defects come from mixing these layers or leaving a convention implicit.

Discount factors are the common language

- ▶ A discount factor $P(0, T)$ is today's value of one currency unit paid with certainty at T .
- ▶ For a deterministic cash flow C_T , $PV = C_T P(0, T)$. For stochastic cash flow X_T , $PV = \mathbb{E}^Q[D(0, T)X_T]$.
- ▶ Zero and forward rates are representations derived from the same discount-factor curve.

Takeaway. Build and validate discount factors first; every later instrument consumes them.

A curve is inferred, not directly observed

- ▶ Markets quote deposits, futures, FRAs, swaps, and OIS rates rather than every maturity's discount factor.
- ▶ Bootstrapping solves instruments sequentially so each quoted instrument reprices on the constructed curve.
- ▶ Interpolation fills unquoted dates; extrapolation policy governs dates beyond the last liquid pillar.

Takeaway. Curve construction is already a calibration problem, even before stochastic models appear.

Day counts and calendars move real money

- ▶ Accrual fractions depend on conventions such as Actual/360, Actual/365, or 30/360.
- ▶ Business-day rules move dates around holidays and weekends; spot lags change effective dates.
- ▶ A one-day scheduling mismatch can alter fixings, coupons, exercise eligibility, and discounting.

Takeaway. Dates and conventions are model inputs, not cosmetic metadata.

An interest-rate swap exchanges coupon systems

- ▶ A payer swap pays fixed and receives floating; a receiver swap does the reverse.
- ▶ Each leg is a signed collection of discounted cash flows with its own schedule and day count.
- ▶ At inception, the par fixed rate makes fixed-leg PV equal floating-leg PV.

Takeaway. The swap NPV is linear in cash flows but nonlinear in the curve used to project and discount them.

The par swap rate is an annuity-weighted forward rate

$$K_{\text{par}} = \frac{P(0, T_0) - P(0, T_n)}{\sum_{i=1}^n \alpha_i P(0, T_i)}, \quad A(0) = \sum_{i=1}^n \alpha_i P(0, T_i).$$

- ▶ The numerator represents the floating-leg value in a single-curve idealization.
- ▶ The denominator $A(0)$ is the fixed-leg annuity, the natural numeraire for swaption formulas.
- ▶ Multi-curve pricing separates forwarding from discounting but retains the annuity intuition.

Takeaway. A swap is the bridge from observable term structures to optionality on rates.

Risk asks how value changes under controlled perturbations

- ▶ DV01 measures value change for a one-basis-point rate move; bucketed DV01 identifies maturity concentration.
- ▶ Vega measures sensitivity to volatility; delta and gamma describe first- and second-order spot exposure.
- ▶ Bump-and-revalue is simple and general, but every bump must preserve market-data consistency.

Takeaway. A reported Greek is incomplete without bump size, units, recalibration rule, and randomness policy.

Options add nonlinear state dependence

- ▶ A call pays $\max(S_T - K, 0)$; a put pays $\max(K - S_T, 0)$.
- ▶ Volatility matters because convex payoffs benefit from dispersion even when the forward is unchanged.
- ▶ Exercise style determines when the holder may choose: European once, Bermudan on discrete dates, American continuously.

Takeaway. Once choice enters the contract, pricing must represent both uncertainty and optimal decisions.

A swaption is an option on a forward swap

- ▶ A payer swaption provides the right to pay fixed; a receiver swaption provides the right to receive fixed.
- ▶ The option's state variable is commonly the forward par swap rate, scaled by the fixed-leg annuity.
- ▶ European swaptions benchmark volatility surfaces and calibrate short-rate models.

Takeaway. Bermudan swaptions add an exercise policy across a sequence of remaining swaps.

Normal and lognormal volatility answer different questions

Bachelier / normal

$S_T = S_0 + \sigma_N W_T$. Rates may cross zero; volatility is quoted in absolute rate units, often basis points.

Black / lognormal

$dS_t/S_t = \sigma_L dW_t$. The state stays positive; volatility is quoted as a percentage.

Takeaway. The quote convention must match the pricing formula and calibration helper.

Calibration translates market prices into model parameters

- ▶ Select liquid instruments whose risks resemble the target product.
- ▶ Minimize a declared error measure: price error, relative price error, or implied-volatility error.
- ▶ Apply bounds, fixed-parameter masks, and convergence criteria to stabilize an underdetermined problem.

Takeaway. A calibrated parameter is meaningful only together with the helper set and objective.

RMS error summarizes calibration dispersion

$$\text{RMS}_{\text{rel}} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{V_i^{\text{model}}}{V_i^{\text{market}}} - 1 \right)^2}.$$

- ▶ Squaring prevents positive and negative residuals from canceling.
- ▶ The relative form gives comparable weight across instruments of different price magnitudes.
- ▶ The repository's XCCY v1 report uses equal helper weights and discloses the exact definition.

Takeaway. One metric never replaces the instrument-by-instrument residual table.

A model is evaluated under a measure

- ▶ The risk-neutral measure is tied to a numeraire such as a domestic money-market account.
- ▶ Changing numeraire changes drifts while leaving correctly priced tradable values invariant.
- ▶ In cross-currency models, the foreign rate drift acquires a quanto correction under the domestic measure.

Takeaway. Writing dynamics without the FX quotation and measure is incomplete.

Trees discretize state; Monte Carlo discretizes paths

- ▶ A tree recombines future states and performs backward induction directly at nodes.
- ▶ Monte Carlo samples full paths and estimates expectations statistically.
- ▶ Trees work especially well for one- or two-factor early-exercise problems; dimensionality grows rapidly.

Takeaway. Monte Carlo scales better in factors, but early exercise requires continuation-value estimation.

Monte Carlo error declines slowly

- ▶ For N independent paths, the standard error is approximately $s/\sqrt{N}$.
- ▶ Halving error therefore requires roughly four times as many paths.
- ▶ Antithetic variates and common random numbers reduce noise without changing the model.

Takeaway. Confidence intervals quantify sampling error, not model risk or exercise-policy bias.

Early exercise is an optimal stopping problem

- ▶ At each exercise date, compare immediate exercise value with conditional continuation value.
- ▶ A tree reads continuation from successor nodes; LSM estimates it by regression on simulated states.
- ▶ The decision must use only information available at that date and must respect event ordering.

Takeaway. A policy fitted and valued on the same paths is optimistically biased; split the samples.

Validation separates implementation from belief

- ▶ Unit tests check deterministic identities and input behavior.
- ▶ Calibration tests confirm quoted instruments reprice within declared tolerances.
- ▶ Martingale, limiting-case, convergence, and independent-benchmark tests interrogate the model.

Takeaway. A successful test suite is evidence, not proof that a model is fit for every trading use.

02 / QuantLib and Python

**Use a library for conventions;
own the economics you extend**

QuantLib is a C++ pricing library with language bindings

- ▶ The C++ core supplies instruments, term structures, models, engines, calendars, solvers, and numerical utilities.
- ▶ Python bindings expose much of that object model while Python orchestrates data, workflows, and presentation.
- ▶ The repository runs QuantLib 1.43 through Python, with NumPy, pandas, and Flask around it.

Takeaway. The binding boundary is productive when native objects fit; it becomes visible when a bespoke joint process is required.

Instrument, engine, and market data are separate objects

- ▶ An instrument describes contractual cash flows and exercise rights.
- ▶ A pricing engine implements a valuation method and reads market handles or model state.
- ▶ Relinkable handles let market data change without rebuilding every instrument.

Takeaway. This separation is the central QuantLib design pattern used throughout the repository.

The global evaluation date is powerful and dangerous

- ▶ `ql.Settings.instance().evaluationDate` defines “today” for many QuantLib objects.
- ▶ Every curve, schedule, fixing check, and option expiry must agree with it.
- ▶ Tests set it explicitly to prevent machine-date-dependent results.

Takeaway. A server must serialize or isolate valuation contexts because this setting is process-global.

Handles express observability and relinking

- ▶ A `QuoteHandle` wraps a market quote; a `YieldTermStructureHandle` wraps a curve.
- ▶ Models and engines subscribe to handles and see updated market objects.
- ▶ This supports bump-and-revalue without changing product definitions.

Takeaway. It also means hidden shared state can surprise concurrent request processing.

Calendars and schedules deserve native library support

- ▶ Generating coupon dates correctly requires holiday calendars, business-day rules, stubs, and termination conventions.
- ▶ QuantLib's `Schedule` makes those rules explicit and testable.
- ▶ The repository uses settlement calendars and explicit spot starts rather than date arithmetic alone.

Takeaway. Reusing native schedule machinery is safer than recreating conventions in custom Python.

Python is the orchestration layer

- ▶ JSON parsing, schema validation, Flask routes, HTML contexts, diagnostics, and CLI behavior are concise in Python.
- ▶ NumPy handles vectorized path simulation and linear algebra for the custom XCCY engine.
- ▶ pandas structures portfolio tables and curve reports.

Takeaway. Python optimizes iteration speed and transparency; it does not erase the need for numerical discipline.

C++ is the native extension layer

- ▶ QuantLib models and engines are designed around C++ inheritance, templates, and shared pointers.
- ▶ A production custom stochastic process can live beside existing term structures and Monte Carlo engines.
- ▶ C++ offers tighter control over memory, parallelism, deterministic numerics, and wrapper exposure.

Takeaway. The cost is build complexity, ABI management, longer iteration cycles, and a larger validation surface.

Python and C++ solve different bottlenecks

Python first

Best for proving dynamics, calibration conventions, cash-flow partitioning, LSM policy design, schemas, and diagnostics.

C++ next

Best when path counts, latency, concurrency, integration policy, or governed library ownership demand it.

Takeaway. Prototype the economics in Python; migrate measured bottlenecks and stable interfaces deliberately.

Bindings are not automatic model composition

- ▶ QuantLib exposes `HullWhiteProcess`, `BlackScholesMertonProcess`, and `StochasticProcessArray`.
- ▶ Putting independent processes into an array correlates shocks but does not derive cross-currency drifts for you.
- ▶ The FX drift must be pathwise $r_d - r_f$ and the foreign factor needs its domestic-measure quanto term.

Takeaway. A syntactically composable process array can still be economically wrong.

QuantLib provides multiple swaption engines

- ▶ Analytic engines include Black, Bachelier, Jamshidian, and G2 approaches.
- ▶ Finite-difference engines cover Hull-White and G2 variants.
- ▶ `TreeSwaptionEngine` handles lattice valuation for a QuantLib Swaption.

Takeaway. Engine availability does not imply support for an arbitrary callable two-currency portfolio.

QuantLib now has native constant-notional XCCY instruments

- ▶ Version 1.43 exposes constant-notional cross-currency swaps, basis swaps, and fixed-versus-floating variants.
- ▶ A discounting constant-notional XCCY engine and cross-currency curve helpers are also available.
- ▶ These are valuable for deterministic cash-flow benchmarks and curve construction.

Takeaway. They do not provide a Bermudan callable hybrid rate/FX engine.

A native instrument is not a native callable engine

- ▶ The upstream source contains XCCY instruments and swaption engines as separate families.
- ▶ Code search finds no callable cross-currency swaption engine joining both families.
- ▶ The one-model tree swaption interface targets a swaption on one swap representation.

Takeaway. The extension therefore reuses primitives while owning joint dynamics and optimal stopping.

Repository architecture keeps Flask thin

- ▶ `app.py` creates the application; the package owns configuration, authentication, routes, and dashboard contexts.
- ▶ `portfolio.py` remains the core single-currency pricing module.
- ▶ The standalone XCCY engine is separate so its numerical workflow is usable without Flask.

Takeaway. Thin delivery layers make model testing and future C++ replacement easier.

JSON makes pricing inputs auditable

- ▶ Versioned market and deal schemas distinguish economics from transport.
- ▶ Unknown fields and unsupported conventions are rejected rather than silently ignored.
- ▶ Normalized input hashing links a result to the exact semantic request.

Takeaway. A reproducible pricer records inputs, model version, software versions, seed, and numerical controls.

Errors should become actionable questions

- ▶ A generic “bad request” does not help a trading-system user complete a deal.
- ▶ The MCP validator maps missing or inconsistent fields to precise questions.
- ▶ Statuses separate READY, NEEDS_INPUT, and INVALID.

Takeaway. Explicit incompleteness is safer than guessing a currency, sign, fixing, or exercise rule.

Tests should mirror the architecture

- ▶ Curve tests reprice bootstrap helpers and inspect zero/forward outputs.
- ▶ Product tests check schedules, identities, marks, sensitivities, and limiting cases.
- ▶ Flask tests verify authentication, routes, rendering, and state updates.

Takeaway. XCCY tests add calibration, martingales, convergence, LSM, and API validation.

A research workbench is not a production risk system

- ▶ The repository favors transparency, deterministic examples, and inspectable intermediate values.
- ▶ It intentionally omits databases, market feeds, distributed workers, XVA, and enterprise entitlements.
- ▶ Production approval would require independent validation, governed data, observability, access control, and release controls.

Takeaway. The documentation states these boundaries instead of implying bank-grade approval.

Technical architecture follows quantitative ownership

- ▶ QuantLib owns standard conventions, curves, helpers, and benchmark engines.
- ▶ Repository Python owns trade normalization, scenario orchestration, UI, and custom hybrid simulation.
- ▶ Deployment owns process isolation, secrets, TLS, service health, and rollback.

Takeaway. Each boundary has different failure modes and therefore different tests.

The core engineering rule is explicitness

- ▶ Explicit dates beat implied schedules; explicit signs beat “payer” ambiguity across systems.
- ▶ Explicit units beat raw numbers; explicit measures beat drift formulas without numerares.
- ▶ Explicit tolerances beat a green status without thresholds.

Takeaway. Explicit limitations make a research model more useful, not less.

03 / March 2025 to September 2025

The sandbox begins with three rates instruments

The repository starts as an empty licensed shell

15 Mar
2025

- ▶ Commit c78135e adds only `.gitignore` and the Apache 2.0 license.
- ▶ GitHub records the public repository creation at 10:14:37 UTC.
- ▶ The initial state establishes ownership and hygiene before any pricing code exists.

Takeaway. Local Git history; GitHub repository metadata.

A standalone Bermudan example appears first

15 Mar
2025

- ▶ Commit 7b3b13c seeds `price_bermudan_swaption.py`.
- ▶ Its later role becomes a thin command-line view over the shared portfolio path.
- ▶ The product ambition – early-exercise rates optionality – is present from day one.

Takeaway. Git history for `price_bermudan_swaption.py`.

Iteration 1 creates the Flask pricing demo

15 Mar
2025

- ▶ Commit 8e17f9f adds 139 lines across the app, pricing module, dependencies, and two templates.
- ▶ The commit message explicitly says the GPT-4.5 output was taken as-is without test runs.
- ▶ The demo prices a swap, European swaption, and purported Bermudan swaption.

Takeaway. Commit 8e17f9f and its source tree.

The first curve is deliberately simple – and flawed

- ▶ One annual deposit helper is created per supplied rate, even though the index is SOFR.
- ▶ The curve uses `PiecewiseLinearZero` with `Actual/360`.
- ▶ No quoted instrument repricing table verifies the construction.

Takeaway. This is sufficient to demonstrate object wiring, not a defensible SOFR OIS curve.

The first swap wires schedules, index, and discounting

- ▶ A five-year payer VanillaSwap uses annual fixed and floating schedules.
- ▶ The fixed leg uses 30/360; the floating leg uses SOFR and Actual/360.
- ▶ A discounting engine values the swap on the constructed curve.

Takeaway. The structure teaches QuantLib composition even though conventions remain simplified.

The first European swaption uses Black volatility

- ▶ A `EuropeanExercise` wraps a date and a `Swaption` wraps the swap.
- ▶ `BlackSwaptionEngine` receives the curve and a flat 1 percent volatility.
- ▶ No market smile, quote convention, or calibration is present.

Takeaway. The engine is a pricing example, not yet a market-calibrated mark.

The initial Bermudan implementation exposes a conceptual error

- ▶ A `BermudanExercise` contains multiple dates.
- ▶ The same `BlackSwaptionEngine` is assigned to the Bermudan instrument.
- ▶ Black's European closed form cannot solve optimal exercise across multiple dates.

Takeaway. Matching an instrument type to an incompatible engine is a classic library-integration failure.

The original swaption underlying also needed separation

- ▶ The first version reused the live spot-start swap for both option types.
- ▶ A European swaption should reference a forward-starting underlying beginning at exercise.
- ▶ Bermudan exercise dates must align with remaining swap cash flows.

Takeaway. Product construction errors can dominate any sophistication in the pricing engine.

Documentation becomes the first reviewed improvement

3 Sep
2025

- ▶ Pull request 1 adds a detailed README and is merged into `main`.
- ▶ The repository becomes easier to install, run, and inspect.
- ▶ Documentation establishes a shared description before deeper model repair.

Takeaway. GitHub PR 1; commits 0bf36bb and 19b73aa.

QuantLib API changes force explicit conventions

3 Sep
2025

- ▶ Pull request 2 fixes `UnitedStates` calendar construction and `Thirty360` initialization.
- ▶ The SOFR index is constructed with a yield-term-structure handle.
- ▶ Flat volatility is passed through a quote handle expected by the modern binding.

Takeaway. Library upgrades reveal implicit assumptions and should be covered by smoke tests.

The Bermudan engine is corrected to a Hull-White tree

- ▶ Commit 6bea9e2 creates a one-factor `ql.HullWhite` model on the curve.
- ▶ `TreeSwaptionEngine(model, 50)` performs backward induction on a short-rate lattice.
- ▶ This matches the discrete exercise structure and replaces the invalid Black engine.

Takeaway. Model and engine compatibility is more important than convenience.

A separate forward-starting swap is introduced

- ▶ The live swap starts today while the option underlying begins one year forward.
- ▶ The European expiry is aligned with that forward start.
- ▶ The Bermudan dates follow the forward start in annual increments.

Takeaway. The change distinguishes current exposure from an option on future swap entry.

Why Hull-White is a natural first rates model

- ▶ It is a one-factor Gaussian short-rate model that fits the initial term structure through a time-dependent drift.
- ▶ Mean reversion controls persistence; volatility controls dispersion.
- ▶ Analytic bond prices and a recombining tree support efficient swaption valuation.

Takeaway. Its simplicity makes assumptions visible, but one factor limits smile and correlation structure.

Hull-White one-factor dynamics

$$dr_t = (\theta(t) - ar_t)dt + \sigma dW_t, \quad P(t, T) = A(t, T)e^{-B(t, T)r_t}.$$

- ▶ $a > 0$ pulls rates toward the time-dependent level implied by $\theta(t)$.
- ▶ σ determines short-rate uncertainty; $B(t, T) = (1 - e^{-a(T-t)})/a$.
- ▶ The affine bond formula is central to trees, analytic swaptions, and later XCCY conditional PVs.

Takeaway. Hull-White separates exact initial-curve fit from a parsimonious stochastic factor.

The early tree remains uncalibrated

- ▶ The model uses default Hull-White parameters rather than a swaption-volatility surface.
- ▶ Fifty time steps are selected without a convergence study.
- ▶ The price is therefore illustrative and cannot claim market consistency.

Takeaway. The next development phase must build curves correctly, add tests, and calibrate volatility.

The early UI is intentionally thin

- ▶ Flask collects a small rate vector and displays a three-row blotter.
- ▶ The web layer and pricing functions are directly coupled.
- ▶ There is no authentication, model diagnostics, persistent state, or risk view.

Takeaway. The small surface is useful for learning but not yet an architecture.

What the first phase taught

- ▶ QuantLib object composition makes a working demo compact.
- ▶ Compact code can conceal convention, product, and engine mismatches.
- ▶ API repairs and a correct Bermudan engine were necessary before adding features.

Takeaway. Tests and calibration become the next logical investments.

The maturity of a pricer is visible in its diagnostics

- ▶ Iteration 1 returns only NPVs.
- ▶ No curve residuals, calibration residuals, schedules, Greeks, or numerical error are exposed.
- ▶ Later work repeatedly adds these observables before adding new model complexity.

Takeaway. Transparency evolves into a design principle for the entire repository.

The project pauses before a major rebuild

- ▶ No recorded commits occur between September 2025 and April 2026.
- ▶ The next changes arrive as runtime repair, OIS bootstrap, and automated tests.
- ▶ The pause separates a proof-of-concept phase from a workstation phase.

Takeaway. Chronology should not infer activity where Git contains no evidence.

04 / 3 April 2026

The demo becomes a tested SOFR workstation

A missing SOFR fixing reveals the valuation boundary

3 Apr
2026

- ▶ Commit ca3d2d0 fixes a runtime failure caused by a floating coupon requiring an unavailable historical fixing.
- ▶ The live swap is moved to a two-business-day spot start.
- ▶ A first Flask smoke-test file is added.

Takeaway. Schedule design and data availability must agree; avoiding accidental historical fixings is an explicit choice.

Spot starting avoids an accidental fixing dependency

- ▶ A swap beginning on the evaluation date can contain an already-fixed or today's overnight observation.
- ▶ The demo has no historical fixing store, so that coupon cannot be projected safely.
- ▶ Advancing by two settlement business days makes all needed fixings prospective.

Takeaway. Production systems should ingest fixings; a sandbox may instead define a clean forward-start boundary.

SOFR curve construction is rebuilt around OIS

3 Apr
2026

- ▶ Commit abfe66f replaces generic annual deposits with an overnight front point and SOFR OIS helpers.
- ▶ The curve becomes PiecewiseLogCubicDiscount.
- ▶ Quoted calibration swaps are repriced and tested.

Takeaway. The market instrument must match the index and collateral economics represented by the curve.

The curve strip grows into a market object

- ▶ Tenors ultimately span 1D, weekly, monthly, and annual pillars out to 30Y.
- ▶ Each quote is normalized from percentage units into decimal rates.
- ▶ The overnight point uses a deposit helper; longer points use `OISRateHelper` with `ql.Sofr`.

Takeaway. A single ordering contract links JSON quotes, labels, helpers, and diagnostics.

Bootstrap logic: solve one maturity at a time

1. Convert each market quote into the instrument helper that owns its conventions.
2. Solve the next unknown curve segment so the new helper has zero NPV.
3. Interpolate log discount factors between solved pillars.
4. Reprice every input instrument and report its residual.

Takeaway. The repricing table is the curve's unit test against its own market definition.

Log-cubic discount interpolation encodes shape assumptions

- ▶ Interpolation is performed on discount factors in logarithmic space, supporting positivity.
- ▶ Cubic behavior produces a smoother curve than piecewise-linear zero rates.
- ▶ Smoothness affects forwards and sensitivities between liquid pillars.

Takeaway. Interpolation choice is part of the model and should be recorded with the curve snapshot.

Discount, zero, and forward views serve different users

- ▶ Discount factors are consumed directly by pricing engines.
- ▶ Continuous Actual/365 zero rates make the identity $P(0, T) = e^{-zT}$ transparent.
- ▶ Daily one-day forwards reveal local interpolation behavior and scenario shape.

Takeaway. The workstation exposes all three rather than treating a curve as one line chart.

Calibration residuals are expected near machine precision

- ▶ Bootstrap helpers define the curve, so their model rates should match input quotes within numerical tolerance.
- ▶ The repository reports market rate, fair rate, and NPV across pillars.
- ▶ A residual spike often indicates a convention, ordering, or curve-range error.

Takeaway. Exact in-sample fit does not guarantee realistic interpolation or out-of-sample instruments.

Repository standards and architecture are documented

3 Apr
2026

- ▶ Commit e13ff48 adds architecture notes and Codex working rules.
- ▶ The mission is defined as a small QuantLib sandbox with Flask and standalone scripts.
- ▶ Quality gates become explicit commands rather than informal expectations.

Takeaway. Architecture documentation changes how later features are integrated and reviewed.

The quality gates make the project reproducible

- ▶ `unittest discover` checks the application and quantitative helpers.
- ▶ `build_SOFR_curve.py` acts as an executable calibration report.
- ▶ A Flask test-client request checks that the app can be created and routed.

Takeaway. A small project benefits from boring, copyable commands more than elaborate infrastructure.

The rates workstation expands dramatically

3 Apr
2026

- ▶ Commit 7eaf671 adds roughly 2,547 lines and introduces the shared base template, dashboard, and trade form.
- ▶ The market surface expands to a swaption normal-volatility matrix.
- ▶ A Bermudan pricing grid and richer state management appear.

Takeaway. The repository shifts from a three-row demo to an interactive quantitative workstation.

State normalization protects the pricing core

- ▶ Defaults originate in JSON and Python constants, then user edits overlay selected fields.
- ▶ Types, ranges, frequencies, directions, and labels are normalized before QuantLib construction.
- ▶ The pricing functions receive one coherent state instead of raw form strings.

Takeaway. Normalization is the boundary between untrusted UI input and quantitative objects.

The workstation carries a full ATM normal-vol matrix

- ▶ Axes are defined explicitly by expiry and underlying-swap tenor labels.
- ▶ Values are stored in basis-point normal volatility, then converted to decimal absolute volatility.
- ▶ Interpolation supplies internal expiries needed by diagonal calibration without inventing on-screen pillars.

Takeaway. The UI shows quote provenance and the model shows which points each trade consumes.

Normal volatility fits the rates regime

- ▶ Bachelier pricing permits negative swap rates and quotes volatility in rate units.
- ▶ A quote of 80 bp means an annualized absolute standard deviation of 0.0080.
- ▶ QuantLib helpers must be told `ql.Normal`; passing the same number as lognormal vol changes meaning.

Takeaway. Unit conversion and volatility type are both contractual fields.

The dashboard becomes a diagnostic surface

- ▶ It shows marks, market inputs, zero curves, forwards, OIS repricing, and a Bermudan grid.
- ▶ Trade editors expose payment and reset frequencies rather than burying conventions.
- ▶ Reset and scenario controls let users distinguish stored market state from perturbed state.

Takeaway. The UI is designed to explain pricing, not merely return a number.

A debug CSV creates an audit seam

- ▶ Each calendar date can carry an input quote, zero rate, and one-day forward.
- ▶ The file allows independent plotting and curve inspection outside Flask.
- ▶ Paths are configured rather than exposed in the page response.

Takeaway. Small audit artifacts make numerical behavior easier to review and reproduce.

Curve risk becomes pointwise

- ▶ Each quoted SOFR node is bumped and the trade repriced.
- ▶ The resulting vector shows where value depends on the curve.
- ▶ A swap should have zero vega cells; a swaption should respond to selected volatility pillars.

Takeaway. Risk-shape tests detect accidental market dependencies better than one aggregate NPV test.

A configurable valuation date replaces the machine clock

- ▶ The default date becomes 10 March 2026 and anchors every schedule and curve.
- ▶ User changes propagate through normalization and reset QuantLib's evaluation date.
- ▶ Deterministic dates make screenshots, workbook comparisons, and tests reproducible.

Takeaway. A production request would carry an explicit as-of timestamp and market snapshot ID.

The curve phase establishes a reusable pattern

- ▶ Build from appropriate helpers.
- ▶ Reprice every input instrument.
- ▶ Expose derived views and sensitivities.

Takeaway. The later USD and EUR XCCY curve bootstraps deliberately reuse this pattern.

05 / 5 April 2026

Swaptions become calibrated models, not flat-vol examples

Curve analytics and calibrated Bermudans arrive

5 Apr
2026

- ▶ Commit 31df798 adds about 2,082 lines across pricing, model documentation, dashboards, routes, and tests.
- ▶ Hull-White calibration replaces default model parameters.
- ▶ G2++ appears as a comparison model and the QuantLib data-model page exposes construction details.

Takeaway. The project begins to separate market fit, model choice, and numerical engine.

European swaptions are calibration instruments

- ▶ A diagonal point is identified by option expiry and remaining swap tenor, such as 2Y x 8Y.
- ▶ The market normal volatility is converted into a helper price.
- ▶ The model price is compared with the same helper after assigning a calibration engine.

Takeaway. This aligns calibration targets with the future swaps relevant to Bermudan exercise.

Jamshidian decomposition provides a Hull-White benchmark

- ▶ A one-factor coupon-bond option can be decomposed into options on zero-coupon bonds.
- ▶ QuantLib's `JamshidianSwaptionEngine` prices calibration helpers analytically under Hull-White.
- ▶ Fast helper valuation makes iterative calibration practical.

Takeaway. The final Bermudan still uses a tree because multiple exercise dates require backward induction.

Mean reversion is fixed; volatility is calibrated

- ▶ The UI supplies Hull-White a as a governed input.
- ▶ The calibration mask fixes a and solves the single constant σ .
- ▶ Levenberg-Marquardt minimizes the selected helper errors under explicit end criteria.

Takeaway. Fixing one parameter avoids trying to identify both a and σ from a narrow diagonal.

The calibration problem is constrained nonlinear least squares

$$\hat{\sigma} = \arg \min_{\sigma > 0} \sum_{i=1}^N w_i (\varepsilon_i(a, \sigma))^2, \quad a \text{ fixed.}$$

- ▶ ε_i may be relative price error or absolute price error.
- ▶ Initial values, bounds, and termination criteria influence robustness.
- ▶ Every helper residual is retained for diagnostics and testing.

Takeaway. Calibration is a reproducible algorithm, not a call to “fit.”

A Bermudan represents a sequence of remaining swaps

- ▶ A 5Y NC2 trade first exercises near year 2 and always ends at the original year-5 maturity.
- ▶ Later decisions enter or cancel progressively shorter remaining swaps.
- ▶ The repository builds labels such as 2Y x 3Y, 3Y x 2Y, and 4Y x 1Y.

Takeaway. Holding final maturity fixed is essential; a fixed underlying tenor would describe a different trade.

Exercise dates are business-day objects

- ▶ The fixed schedule defines underlying start dates and final maturity.
- ▶ Exercise is placed one business day before the corresponding underlying start.
- ▶ QuantLib calendars adjust every date consistently.

Takeaway. Tests assert the exact 2028–2031 dates for the benchmark schedule.

The diagonal helper strip follows the call schedule

- ▶ Each exercise date maps to the remaining swap tenor.
- ▶ If an expiry is absent from the visible matrix, the repository interpolates between governed neighboring expiries.
- ▶ Source market points are retained and shown in the UI.

Takeaway. Calibration lineage connects each model residual to an exercise opportunity.

Tree pricing performs backward induction

- ▶ The short-rate state is discretized on a recombining lattice.
- ▶ At the final exercise date, exercise and continuation values are known at each node.
- ▶ Moving backward, the engine takes the larger holder value at each eligible node.

Takeaway. Discounted transition expectations propagate the optimal value to today.

The local exercise rule

$$V(t_i, x) = \max(E(t_i, x), C(t_i, x)), \quad C(t_i, x) = \mathbb{E}[D_{i,i+1} V(t_{i+1}, X_{i+1}) \mid X_i = x].$$

- ▶ The state x is one-dimensional under Hull-White 1F.
- ▶ Node values share successor states, producing a recombining tree.
- ▶ Numerical time steps control approximation error between event dates.

Takeaway. A Bermudan price includes both cash-flow value and the value of a stopping policy.

Tree steps are a numerical parameter

- ▶ The repository scales steps with final maturity and enforces a minimum of 50.
- ▶ Too few steps distort exercise and curve fitting on the lattice.
- ▶ More steps increase runtime and may expose date-grid alignment effects.

Takeaway. A production model needs an explicit tree-step convergence study.

G2++ adds a second Gaussian rate factor

- ▶ Two mean-reverting factors allow richer yield-curve movements than Hull-White 1F.
- ▶ Volatilities σ and η are calibrated while mean reversions and factor correlation are fixed in this workbench.
- ▶ QuantLib provides a G2 swaption calibration engine and tree-compatible short-rate model machinery.

Takeaway. The comparison reveals model risk without replacing the transparent Hull-White default.

G2++ decomposes the short rate into two factors

$$r_t = \varphi(t) + x_t + y_t, \quad dx_t = -ax_t dt + \sigma dW_t^x, \quad dy_t = -by_t dt + \eta dW_t^y, \quad dW_t^x dW_t^y = \rho dt.$$

- ▶ $\varphi(t)$ fits today's term structure.
- ▶ Two factors can represent level and slope-like movements.
- ▶ More parameters improve flexibility but complicate identification and validation.

Takeaway. Additional factors should answer a measured model deficiency, not merely add sophistication.

Calibration method becomes user-visible

- ▶ The trade stores relative-price-error or price-error selection.
- ▶ QuantLib helper enums are displayed rather than hidden behind an opaque label.
- ▶ The model view reports fixed and calibrated parameters separately.

Takeaway. A user can explain why two calibrations differ instead of seeing only two NPVs.

The model page documents the object graph

- ▶ It names instruments, factories, signatures, purposes, dependencies, fields, and enumerations.
- ▶ Schedules and model parameters become inspectable educational objects.
- ▶ Research references are linked to the actual repository implementations.

Takeaway. Documentation is generated from the same normalized state used by pricing.

Workbook alignment adds an external benchmark

- ▶ A second Bermudan trade is seeded with the spreadsheet reference economics.
- ▶ The test expects a mark near USD 1.414 million within a USD 20,000 tolerance.
- ▶ The schedule, surface interpolation, model, and tree must work together to meet the benchmark.

Takeaway. A benchmark test is stronger than checking only that NPV is positive.

Exercise probabilities remain a known tree limitation

- ▶ The UI lists the call schedule and calibration sources.
- ▶ QuantLib's tree swaption engine returns value but does not expose the exercise probability profile used by this app.
- ▶ The repository says “not exposed” rather than inventing statistics.

Takeaway. The later XCCY LSM engine records exercise counts explicitly because it owns the policy.

Sensitivity maps reveal calibration dependency

- ▶ The European swaption responds to one selected matrix pillar.
- ▶ The Bermudan responds to the diagonal and any interpolation source points.
- ▶ Mean-reversion bumps rebuild and recalibrate the model.

Takeaway. Tests assert the exact nonzero volatility labels, turning data lineage into executable behavior.

The calibrated model still has deliberate simplifications

- ▶ A single constant volatility parameter compresses an entire diagonal strip.
- ▶ Mean reversion is fixed rather than statistically identified.
- ▶ The one-factor model cannot reproduce all swaption-surface shapes simultaneously.

Takeaway. The workbench surfaces residuals so users can see the cost of parsimony.

Single-currency Bermudans are a tree problem here

- ▶ One curve economy and one short-rate state keep lattice dimensionality manageable.
- ▶ QuantLib already has the instrument, model, calibration helpers, and tree engine.
- ▶ This native path is retained unchanged when cross-currency work begins.

Takeaway. A new XCCY engine should not destabilize a validated benchmark path.

On-demand risk follows calibrated pricing

5 Apr
2026

- ▶ Commit aa50775 adds roughly 688 lines for trade risk analytics and richer trade pages.
- ▶ Curve, volatility, and model-parameter bumps are computed when requested.
- ▶ The dashboard stays responsive by avoiding unconditional scenario expansion.

Takeaway. Risk is treated as a controlled workflow around the same pricing functions.

Recalibration policy defines a Greek

- ▶ A curve or volatility bump may change both direct valuation inputs and calibrated model parameters.
- ▶ Holding parameters fixed measures one risk; full recalibration measures another.
- ▶ The repository's diagnostics state which convention is used.

Takeaway. Comparisons are meaningful only when bump and calibration policies agree.

The Bermudan phase prepares the XCCY extension

- ▶ It establishes schedule semantics, diagonal calibration, residual reporting, and explicit exercise dates.
- ▶ It demonstrates native QuantLib backward induction in a low-dimensional setting.
- ▶ It also exposes what the native engine does not report or generalize.

Takeaway. The later hybrid model keeps the economics and replaces only the state evolution and stopping engine.

A calibrated tree is a benchmark, not a universal engine

- ▶ Its strengths are speed, deterministic backward induction, and native QuantLib integration.
- ▶ Its boundary is the one-economy swaption instrument and low-dimensional short-rate state.
- ▶ Cross-currency optionality adds a second rate factor, FX, quanto drift, and two-currency cash flows.

Takeaway. Recognizing that boundary prevents forcing the wrong abstraction onto the next product.

06 / 5–8 April 2026

Equity cliquets broaden the numerical toolkit

The workbench aligns to a broader reference portfolio

7 Apr
2026

- ▶ Commit 7e08907 merges pull request 3 and changes 42 files with about 4,363 insertions.
- ▶ Default market and trade JSON files become explicit data sources.
- ▶ The portfolio now includes a swap, European swaption, two Bermudans, and an SPX cliquet.

Takeaway. The repository becomes a multi-asset educational workbench rather than a rates-only demo.

A cliquet is a strip of forward-start options

- ▶ The underlying return is measured independently over successive reset periods.
- ▶ Each period applies local strike or moneyness terms, then discounted values are summed.
- ▶ The contract is path-dependent through reset levels even when period options admit analytic decomposition.

Takeaway. The structure teaches how product decomposition can avoid unnecessary full-path Monte Carlo.

A simple cliquet payoff decomposition

$$\Pi_T = Q \sum_{j=1}^m \max \left(\omega \left(\frac{S_{T_j}}{S_{T_{j-1}}} - K_j \right), 0 \right), \quad \omega \in \{+1, -1\}.$$

- ▶ Each reset creates a forward-start option for the next period.
- ▶ Quantity Q , call/put sign ω , reset frequency, and maturity define the strip.
- ▶ Contract-specific caps, floors, and ratchets would change this generic formula.

Takeaway. Path dependence does not automatically imply one monolithic Monte Carlo engine.

The equity market context is intentionally flat

- ▶ Inputs are SPX spot, a flat dividend yield, a flat risk-free curve mapping, and flat Black volatility.
- ▶ The calendar and day count are explicit.
- ▶ The simplified surface isolates payoff mechanics and Greeks.

Takeaway. A production cliquet would require term/skew surfaces, corporate actions, and convention governance.

Black-Scholes supplies period unit values

- ▶ Each forward-start period can be reduced to a normalized Black formula under flat deterministic inputs.
- ▶ Discounting and quantity scale the unit option value.
- ▶ Analytic delta, gamma, and vega can be aggregated across periods.

Takeaway. The implementation uses transparent formulas where QuantLib's direct cliquet engine coverage is limited.

Reset schedules are part of the payoff

- ▶ Maturity and reset frequency determine observation dates.
- ▶ The final short stub must be handled rather than discarded.
- ▶ Business-day adjustment changes time fractions and therefore forward and volatility scaling.

Takeaway. Tests cover periodic decomposition and schedule consistency.

Analytic identities become powerful tests

- ▶ The total cliquet NPV should equal the sum of discounted reset-period components.
- ▶ Call-put parity can be tested period by period under matching assumptions.
- ▶ Greeks can be compared with finite-difference approximations.

Takeaway. Identity tests catch sign and scaling defects that a single regression mark may miss.

A payoff profile still benefits from Monte Carlo

- ▶ Simulation illustrates the distribution of realized cliquet payoffs across paths.
- ▶ The analytic NPV remains the benchmark under matching assumptions.
- ▶ Comparing analytic and simulated views teaches sampling error without making MC the production method.

Takeaway. The dashboard separates expected value, scenarios, and payoff distribution.

Spot-vol scenarios expose nonlinear interaction

- ▶ A two-dimensional grid varies spot and volatility around the base market.
- ▶ Convexity makes the response non-planar.
- ▶ Scenario grids are intuitive, while Greeks summarize only local behavior.

Takeaway. The trade page offers both so users see approximation limits.

Delta, gamma, and vega need stable definitions

- ▶ Delta is the derivative with respect to spot in currency per index point.
- ▶ Gamma is the rate of change of delta and is sensitive to bump size.
- ▶ Vega must state whether it is per one volatility point or per unit volatility.

Takeaway. The repository tests analytic and bump-based values under declared units.

Equity work changes the architecture, not only the blotter

- ▶ Market state now contains both rates and equity blocks.
- ▶ Trade editors and dashboard context must dispatch by product family.
- ▶ Risk tables need product-specific relevant and irrelevant cells.

Takeaway. The shared application remains one Flask process despite heterogeneous pricing logic.

A product factory must reject nonsensical combinations

- ▶ Option type, moneyness, reset frequency, maturity, and quantity are normalized.
- ▶ Reset frequency cannot exceed maturity.
- ▶ Unknown or malformed values fall back only where the workbench explicitly defines defaults.

Takeaway. Input validation protects formulas from silently pricing a different product.

Research notes are linked to implementation choices

- ▶ The repository records references for swaption pricing and equity cliquets.
- ▶ Notes distinguish what QuantLib supplies from formulas implemented explicitly.
- ▶ The model page links literature to the code paths users can inspect.

Takeaway. This turns documentation into a guide for challenge and extension.

The cliquet demonstrates selective QuantLib use

- ▶ QuantLib remains valuable for dates, curves, discounting, and market handles.
- ▶ The repository implements transparent Black-style period formulas directly.
- ▶ Not every product needs a bespoke QuantLib C++ instrument before it can be studied rigorously.

Takeaway. The same hybrid ownership philosophy later guides the XCCY pricer.

Risk analytics become on demand

- ▶ Full scenario expansion can dominate page latency.
- ▶ The UI requests detailed risk only when the user opens a trade analysis.
- ▶ The same normalized market context is reused across base and bumped valuations.

Takeaway. Performance is managed at the workflow boundary before introducing distributed infrastructure.

Common random numbers foreshadow XCCY diagnostics

- ▶ For Monte Carlo Greeks, using identical random draws for up and down bumps reduces difference noise.
- ▶ The equity example teaches the idea even when analytic formulas are available.
- ▶ The XCCY diagnostics later make common seeds an explicit method field.

Takeaway. Variance reduction is most valuable when estimating differences rather than levels.

The equity addition broadens model-risk vocabulary

- ▶ Rates models emphasize curve fit, mean reversion, and exercise policy.
- ▶ Equity options emphasize lognormal dynamics, dividend yield, smile, convexity, and path dependence.
- ▶ The later XCCY model combines both vocabularies: short rates plus lognormal FX.

Takeaway. Cross-asset work becomes a synthesis rather than a disconnected feature.

Tests become a quantitative narrative

- ▶ A test name states the intended identity or monotonicity.
- ▶ Inputs and tolerances make the claim executable.
- ▶ Failures identify whether the issue is product logic, calibration, numerics, or UI integration.

Takeaway. The lecture can therefore use tests as evidence of what the model promises.

The workbook-aligned phase closes the prototype gap

- ▶ Market and trade inputs move from hard-coded lists into versionable JSON.
- ▶ Reference marks, risk shapes, and UI views are tested.
- ▶ CI and deployment assets arrive in the same broad commit.

Takeaway. The project is ready to be treated as an application, not just a notebook substitute.

07 / 7–14 April 2026

The quantitative sandbox becomes an operated web application

The application factory separates creation from execution

- ▶ `create_app()` loads configuration, registers routes, and returns a Flask application.
- ▶ `app.py` is the local entrypoint; `wsgi.py` is the production entrypoint.
- ▶ Tests can create isolated apps with temporary configuration.

Takeaway. This pattern prevents server startup side effects from contaminating model tests.

Authentication protects the workstation boundary

- ▶ The root page is a login screen and quantitative routes require a session.
- ▶ Credentials and the Flask secret come from environment configuration.
- ▶ Tests use explicit temporary credentials and disable secure cookies locally.

Takeaway. The repository does not commit production secrets.

Routes remain delivery adapters

- ▶ Routes translate HTTP form or JSON input into normalized state.
- ▶ Pricing remains in `portfolio.py` or the standalone XCCY module.
- ▶ Context builders turn quantitative outputs into view-specific structures.

Takeaway. Keeping Flask thin lets CLI, tests, MCP, and future C++ clients reuse the same economics.

Server-rendered templates fit the project's scale

- ▶ Jinja templates produce the dashboard, trade forms, model page, and later XCCY lab.
- ▶ There is no separate React build, API gateway, or database.
- ▶ The architecture favors a single host and minimal infrastructure.

Takeaway. This is a conscious deviation, not an unfinished split frontend.

Session state supports a demonstration workflow

- ▶ User-edited market and trade inputs live in the Flask session.
- ▶ Reset returns to deterministic defaults loaded from JSON.
- ▶ A realtime tick perturbs the demo state and returns updated analytics.

Takeaway. Production multi-user state would require stronger persistence and audit semantics.

UI and deployment documentation are consolidated

11 Apr
2026

- ▶ Commit 65a2acb changes 31 files, adds a handoff, deployment guide, screenshots, and extensive UI refinements.
- ▶ Local timestamps show the long-lived pricing module continuing to evolve after this commit.
- ▶ Operational documentation becomes part of the delivered system.

Takeaway. A model is usable only when users can install, run, diagnose, and recover it.

CI encodes repeatable confidence

- ▶ GitHub Actions creates Python 3.13, installs requirements, and runs selected test modules.
- ▶ GitHub records successful CI runs for the April 2026 main-branch commits.
- ▶ The local full suite is broader than the historical release-gate subset.

Takeaway. A green pipeline is meaningful only when its selected tests match current critical paths.

DigitalOcean deployment is a simple three-tier stack

- ▶ Apache terminates TLS and proxies requests.
- ▶ Gunicorn binds the Flask app only on `127.0.0.1:8008`.
- ▶ systemd owns process startup and restart under `www-data`; the application root is `/var/www/html/tgir_quantlib_tools`.

Takeaway. One approximately USD 7-per-month Ubuntu droplet keeps the hobby affordable while preserving explicit process boundaries.

Production request path



- ▶ Secrets remain in a mode-600 server-side `.env`; deployment builds a fresh `.venv` at its final path.
- ▶ Health checks test `127.0.0.1:8008/health` and `https://quant.tglauner.com/health`.

Takeaway. About USD 7 per month is enough to keep a serious quantitative hobby continuously reachable behind production-style controls.

Deployment failures expose architecture assumptions

- ▶ GitHub shows successful CI but repeated failed DigitalOcean workflows in April 2026.
- ▶ Automated delivery requires `DO_HOST`, `DO_USER`, `DO_SSH_KEY`, `DO_APP_ROOT`, and `DO_HEALTHCHECK_URL`.
- ▶ A prebuilt virtualenv could not be moved because executable shebangs contained absolute paths; rollback succeeded.

Takeaway. The corrected release rebuilt its virtualenv at the final path and passed health checks.

Rollback is part of deployment design

- ▶ The live app is preserved before replacement.
- ▶ A failed service health check restores the prior directory and restarts systemd.
- ▶ The successful August deployment retained a timestamped rollback copy.

Takeaway. Recoverability matters more than pretending a release cannot fail.

An IBKR comparison dashboard is added

13 Apr
2026

- ▶ Commit 711ce0d adds more than one thousand lines centered on a second dashboard.
- ▶ The interface compares workbench values with an execution-platform-style view.
- ▶ Routing and tests expand while the pricing core remains shared.

Takeaway. Multiple views can consume one normalized pricing state without duplicating model code.

Presentation changes should not change economics

- ▶ Subsequent commits tighten monitor layout, realtime ticks, and reset-control styling.
- ▶ Commit names 754707f, 297e02c, and d3b45a4 are primarily interface refinements.
- ▶ Tests protect core marks and route behavior while CSS and templates evolve.

Takeaway. Chronology distinguishes visual iterations from model changes.

A dashboard bug demonstrates signature drift

- ▶ In August, `_forward_rate_chart` required regular and scenario points but one caller supplied only one argument.
- ▶ The dashboard returned HTTP 500 after login.
- ▶ The call site was repaired and smoke tests were expanded.

Takeaway. Function signatures shared across UI paths need direct route tests.

Local timestamps refine the August sequence

- ▶ The XCCY quantitative summary was born 14 August.
- ▶ The standalone pricer, JSON deal/market files, schemas, and core tests were born 15 August.
- ▶ Diagnostics and MCP followed later on 15 August; validation and web work followed 16–17 August.

Takeaway. Because these landed in one Git commit, filesystem times provide supporting – not authoritative – ordering.

Configuration keeps paths and secrets outside pricing code

- ▶ Config values identify market, deal, result, debug, authentication, and cookie behavior.
- ▶ Tests override paths with temporary directories.
- ▶ Production reads the same keys from the server environment.

Takeaway. The design avoids hard-coded personal filesystem paths in runtime logic.

Thread safety matters around QuantLib globals

- ▶ The evaluation date and relinkable handles can be process-global or shared.
- ▶ Concurrent valuations with different as-of dates can interfere if not isolated.
- ▶ A production C++ or Python service should use serialized contexts or process isolation.

Takeaway. The current demo uses a small Gunicorn process model and is not a high-throughput risk grid.

The architecture optimizes for inspectability

- ▶ Source files remain direct and readable rather than distributed across services.
- ▶ JSON snapshots, CSV exports, HTML diagnostics, CLI scripts, and tests expose intermediate states to Tim, Codex, and Claude alike.
- ▶ The cost is limited concurrency and no database-backed audit store.

Takeaway. For a research workbench, this tradeoff enables two agents to build, review, extend validation, and reproduce each other's evidence.

By mid-April, the platform is ready for a cross-asset challenge

- ▶ Curves, rates options, equity options, calibrated trees, risk, tests, authentication, and deployment all exist.
- ▶ The next requested product combines two curves, two short-rate factors, FX, and early cancellation.
- ▶ The existing architecture offers reusable primitives but not a native engine.

Takeaway. Tim directs the August challenge; Codex begins with rigorous specifications before source implementation, creating a stable target for Claude's later review and validation extensions.

08 / 14–15 August 2026

**Does QuantLib natively price
callable XCCY swaps?**

The work begins as specification, not code

14 Aug
2026

- ▶ A quantitative model summary is created before the standalone implementation.
- ▶ The product, measure, dynamics, calibration, Monte Carlo, and LSM choices are written down.
- ▶ A separate trading-system integration specification keeps external API concerns distinct.

Takeaway. Writing the contract first exposes ambiguities that code would otherwise hard-wire.

The target deal is economically specific

- ▶ Ten-year constant-notional EUR/USD swap, non-callable for two years, then annually cancellable through year nine.
- ▶ The holder receives quarterly compounded EUR ESTR and pays semiannual USD fixed at 4%.
- ▶ USD reporting and collateral, USD 100 million versus EUR 90.909 million, with initial and final exchanges.

Takeaway. A zero-fee call cancels remaining cash flows after due payments under declared event ordering.

“Callable” is not a complete product definition

- ▶ The right may cancel an existing swap or enter a remaining swap.
- ▶ The exercise holder, sign perspective, notice date, effective date, and settlement currency matter.
- ▶ Same-day coupons and notionals must be partitioned by event order.

Takeaway. Version 1 supports CANCEL_REMAINING_SWAP with notice equal to decision date.

The exercise decision separates common and exclusive cash flows

- ▶ Let A_i contain flows that survive either decision through the effective date.
- ▶ Let J_i be exercise-only settlement value and Y_i the cancellable continuation tail.
- ▶ Compare J_i with $\mathbb{E}[Y_i \mid \mathcal{F}_{t_i}]$; add A_i after the decision.

Takeaway. This prevents already-due or delayed common cash flows from disappearing at cancellation.

The model needs three correlated risk factors

- ▶ One Hull-White factor drives USD rates.
- ▶ One Hull-White factor drives EUR rates.
- ▶ One lognormal factor drives EURUSD quoted as USD per EUR.

Takeaway. A full positive-semidefinite 3-by-3 correlation matrix couples all Brownian shocks.

Why not extend the single-currency lattice directly?

- ▶ A three-dimensional lattice grows rapidly in nodes and transition structure.
- ▶ Two currencies introduce path-dependent conversion and domestic-measure drift corrections.
- ▶ The existing `TreeSwaptionEngine` accepts one short-rate model and a swaption, not an arbitrary callable XCCY portfolio.

Takeaway. Monte Carlo scales more naturally in factors and supports explicit pathwise cash flows.

QuantLib 1.43 does support deterministic XCCY instruments

- ▶ Bindings expose constant-notional cross-currency swaps, basis swaps, and fixed-versus-floating swaps.
- ▶ A discounting constant-notional XCCY engine provides deterministic present-value benchmarks.
- ▶ Cross-currency rate helpers support curve construction.

Takeaway. These primitives are reused conceptually even though the callable engine is custom.

QuantLib 1.43 supports the single-currency option pieces

- ▶ Hull-White and G2 short-rate models are native.
- ▶ Swaption helpers and Jamshidian engines support calibration.
- ▶ Tree and finite-difference swaption engines support Bermudan exercise on native swaption instruments.

Takeaway. These components validate each currency separately but do not assemble the hybrid callable.

QuantLib exposes multi-process simulation primitives

- ▶ `StochasticProcessArray` can correlate component process innovations.
- ▶ `GaussianMultiPathGenerator` can generate multiple state paths.
- ▶ Hull-White and Black-Scholes-Merton processes are individually available.

Takeaway. A generic process array does not automatically impose the cross-currency measure change.

The missing economics are in the drifts

$$\begin{aligned}dr_d &= [\theta_d(t) - a_d r_d]dt + \sigma_d dW_d, \\dr_f &= [\theta_f^{(f)}(t) - a_f r_f - \rho_{fS} \sigma_f \sigma_S(t)]dt + \sigma_f dW_f, \\d \log S &= [r_d - r_f - \frac{1}{2} \sigma_S^2(t)]dt + \sigma_S(t) dW_S.\end{aligned}$$

Takeaway. Correlating independent native processes is insufficient; the domestic-measure drift system must be implemented jointly.

The quanto sign depends on declared conventions

- ▶ Domestic is USD, foreign is EUR, and S is USD per EUR.
- ▶ The pricing measure uses the USD money-market numeraire.
- ▶ Under those conventions, the EUR short-rate drift receives $-\rho_{fS}\sigma_f\sigma_S$.

Takeaway. Reversing the FX quotation or changing numeraire changes the derived drift.

The foreign effective curve is a collateralized object

- ▶ USD collateral, EUR cash flows, spot FX, and FX forwards define an effective EUR discount curve under USD collateral.
- ▶ The identity $S_0 P_f^*(0, T) = \mathbb{E}^{Q_d}[D_d(0, T) S_T]$ anchors the measure-consistent interpretation.
- ▶ The EUR forecast curve is mapped through a deterministic forecast/discount basis.

Takeaway. This version has no stochastic cross-currency basis factor.

Native support is a spectrum, not a yes/no answer

- ▶ Native: dates, calendars, schedules, OIS helpers, curves, swaption helpers, Hull-White calibration, analytic benchmarks.
- ▶ Partly native: XCCY deterministic instruments and generic multi-process utilities.
- ▶ Not native as one engine: the joint domestic-measure process, callable XCCY cash flows, and LSM stopping policy.

Takeaway. The implementation boundary is selected component by component.

Extension path: Python reference versus C++ engine

Python reference

Fast to inspect and modify; NumPy vectorization; direct JSON/MCP/web integration; ideal for validating signs, timing, and regression.

C++ production

Native QuantLib types; lower latency; tighter memory and concurrency control; reusable compiled engine behind C++ APIs.

Takeaway. Economics and tests should stabilize before translating the hot path.

A C++ extension would define a coherent joint process

- ▶ State could be $(x_d, x_f, \log S, \int r_d du)$ with exact or controlled transitions.
- ▶ The process would own measure-consistent drift and integrated covariance.
- ▶ A callable cash-flow instrument or engine would own event ordering and policy evaluation.

Takeaway. Bindings would expose versioned inputs and structured results without leaking internal handles.

Python remains valuable even after C++ migration

- ▶ It can generate benchmark snapshots and regression vectors.
- ▶ It supports calibration investigation, scenario exploration, and model-validation notebooks.
- ▶ The Flask/MCP delivery layers can call a compiled library without changing their contracts.

Takeaway. Independent implementations in two languages strengthen comparison when ownership is clear.

The v1 scope is intentionally narrower than a trading library

- ▶ Constant notionals, USD collateral, one exercise holder, no partial calls, and no collateral switching.
- ▶ Rate volatility is one-piece constant per currency; FX volatility is ATM and piecewise constant.
- ▶ Basis is deterministic; no stochastic volatility, jumps, XVA, or funding adjustment.

Takeaway. Explicit exclusions keep the first model testable.

The decision: Monte Carlo plus two-pass LSM

- ▶ Monte Carlo handles two rates, FX, correlations, discounting, and pathwise currency conversion.
- ▶ Backward regression learns continuation value on independent training paths.
- ▶ A frozen policy is applied forward to a separately seeded pricing sample.

Takeaway. The resulting estimate is a lower-bound policy value with reported sampling error.

The specification becomes an executable reference engine

15 Aug
2026

- ▶ Local birth times place the standalone pricer at 19:19, followed by deal/market JSON, schemas, and tests.
- ▶ The implementation stays separate from `portfolio.py`.
- ▶ QuantLib is reused for market construction and calibration; NumPy owns the hybrid simulation.

Takeaway. The separation preserves existing single-currency prices while enabling rigorous experimentation.

09 / 15–17 August 2026

Extending QuantLib with an explicit Python hybrid engine

The input validator is the first pricing component

- ▶ Schemas require versioned market and deal objects.
- ▶ Semantic checks reject unsupported currencies, FX quote direction, exercise style, correlations, schedules, and nonpositive numerics.
- ▶ Unknown fields are rejected to prevent silent contract drift.

Takeaway. The engine refuses to price economics it cannot represent faithfully.

The correlation matrix must be admissible

- ▶ Each off-diagonal correlation lies in $[-1, 1]$.
- ▶ The symmetric matrix must be positive semidefinite.
- ▶ The web editor computes the determinant and disables repricing when the matrix is invalid.

Takeaway. The simulator uses an eigenvalue-based covariance root robust to semidefinite cases.

Three curves serve distinct jobs

- ▶ USD SOFR is the domestic discount curve.
- ▶ EUR ESTR is the foreign forecast curve for floating coupons.
- ▶ FX forwards imply an effective EUR curve consistent with USD collateral.

Takeaway. The separation prevents a foreign forecast curve from being mistaken for the cross-currency discount curve.

FX forwards imply the effective foreign curve

$$F(0, T) = S_0 \frac{P_d(0, T)}{P_f^*(0, T)} \implies P_f^*(0, T) = S_0 \frac{P_d(0, T)}{F(0, T)}.$$

- ▶ S and F are both quoted USD per EUR.
- ▶ Interpolated forward outrights carry collateralized cross-currency basis information.
- ▶ The result report verifies every supplied forward against the constructed effective curve.

Takeaway. Cross-currency basis enters v1 deterministically through the effective foreign curve.

Each rate model is calibrated independently

- ▶ USD and EUR diagonal normal swaptions use their corresponding forecast/discount economy.
- ▶ Mean reversion rates are supplied as 3% for USD and 4% for EUR.
- ▶ One constant sigma per currency is solved with QuantLib helpers and Jamshidian engines.

Takeaway. The stored result reports approximately 0.885% USD sigma and 0.772% EUR sigma.

Constant rate volatility is a deliberate v1 compromise

- ▶ Eight diagonal helpers cannot all be matched exactly by one sigma.
- ▶ The weighted RMS relative price error summarizes the residual strip.
- ▶ Calibration limits reject excessive aggregate error.

Takeaway. Piecewise rate volatility is a future model version, not an undocumented tweak.

FX volatility is calibrated in the hybrid variance

- ▶ Market targets are ATM Black volatilities at 1Y, 2Y, 3Y, 5Y, 7Y, and 10Y.
- ▶ With stochastic rates, the diffusion coefficient is not blindly copied from the Black quote.
- ▶ Bucket sigmas are solved so total hybrid log-FX variance reproduces the target maturity variance.

Takeaway. The calibration reports re-implied vol and absolute residual at every tenor.

Hybrid log-FX variance contains rate and FX terms

$$\text{Var}[\log S_T] = \int_0^T \|\sigma_S(u)e_S + \sigma_d B_d(u, T)e_d - \sigma_f B_f(u, T)e_f\|_R^2 du.$$

- ▶ R is the instantaneous correlation matrix.
- ▶ Rate shocks enter terminal FX through integrated short-rate differentials.
- ▶ Bucket calibration solves for nonnegative σ_S consistent with accumulated variance.

Takeaway. An ATM Black quote is a target price statistic, not automatically the hybrid FX diffusion.

Cash-flow planning happens before simulation

- ▶ QuantLib calendars and schedules generate EUR quarterly and USD semiannual periods.
- ▶ The plan records fixing/accrual boundaries, payment dates, currencies, signs, notionals, and exercise dates.
- ▶ Initial/final exchanges and payment lags are represented explicitly.

Takeaway. Simulation consumes a deterministic event plan rather than reconstructing contracts path by path.

The time grid contains every economically relevant event

- ▶ Coupon boundaries, payments, notional exchanges, and call dates are inserted exactly.
- ▶ Intermediate dates cap each time step at one month by default.
- ▶ Piecewise FX-vol bucket ends also enter the grid.

Takeaway. A controlled grid prevents accidental time aggregation across cash-flow or parameter discontinuities.

Exact OU transitions reduce discretization bias

- ▶ Hull-White state factors use exact conditional mean reversion over each interval.
- ▶ The transition matrix applies $e^{-a\Delta t}$ to each rate state.
- ▶ Integrated covariance includes rate states, log FX, and the domestic discount integral.

Takeaway. Only deterministic coefficient integration remains numerical within each step.

One exact Gaussian step

$$Z_{t+h} = M(h)Z_t + m(t, h) + L(t, h)\varepsilon, \quad \varepsilon \sim N(0, I), \quad LL^\top = Q(t, h).$$

- ▶ Z contains rate factors, log FX, and integrated domestic short rate.
- ▶ M captures exact OU decay; Q integrates all correlated diffusion kernels.
- ▶ Eigen factorization handles covariance matrices close to singularity.

Takeaway. Exact conditional Gaussian structure is used wherever the chosen model permits it.

The log update preserves positive FX

- ▶ The state is $X = \log S$, not S itself.
- ▶ Its drift includes $r_d - r_f - \frac{1}{2}\sigma_S^2$.
- ▶ Exponentiation guarantees positive simulated EURUSD.

Takeaway. The explicit Ito term prevents the common error of using the spot drift directly in log space.

Floating EUR coupons use a deterministic basis mapping

- ▶ EUR rate uncertainty comes from the EUR Hull-White factor.
- ▶ The initial ratio between forecast and effective discount curves is frozen deterministically.
- ▶ Conditional forecast bonds derive floating accruals without simulating every overnight fixing.

Takeaway. This is an explicit approximation with zero lookback and lockout in v1.

Pathwise valuation respects currency and discounting

- ▶ USD cash flows are discounted by the simulated USD money-market factor.
- ▶ EUR cash flows are converted at pathwise EURUSD and discounted domestically.
- ▶ Leg PVs, non-callable PV, and interval PVs between call dates are retained separately.

Takeaway. A deterministic static-curve benchmark checks the non-callable cash-flow plan.

LSM state variables explain continuation value

- ▶ Standardized states include both rate factors, log FX, and remaining-leg economics.
- ▶ The basis contains a constant, linear terms, squares, and pairwise cross-products.
- ▶ Mean and scale are frozen with the regression coefficients.

Takeaway. Rank, condition number, ridge, residual RMS, and training count are reported per call date.

Backward training learns when continuation is negative

1. Start with the realized cancellable tail after the last exercise date.
2. Regress the discounted continuation realization on current state variables.
3. For zero-fee cancellation, exercise when fitted continuation is below zero.
4. Replace future value by zero on exercised training paths and move backward.

Takeaway. Cancellation is valuable because it removes a negative tail, not because it pays a positive intrinsic amount.

Forward policy application produces an honest price sample

- ▶ A new seed generates independent pricing paths.
- ▶ At each call date, the frozen basis and coefficients determine exercise for alive paths.
- ▶ The first exercise stops only the cancellable tail; prior and common flows remain in value.

Takeaway. The sample average and standard error are computed without refitting.

The sample result exposes value decomposition

- ▶ The stored snapshot reports callable NPV around USD 8.34 million and non-callable NPV around USD 0.69 million.
- ▶ The embedded cancellation option is about USD 7.65 million in that stored run.
- ▶ Leg PVs and a static-curve benchmark support reconciliation.

Takeaway. These values are illustrative and depend on the stored market, correlations, seed, and model version.

Exercise statistics make the policy observable

- ▶ Eight annual call dates report alive paths, exercises, unconditional probability, and conditional probability.
- ▶ The stored run leaves roughly 51% of paths unexercised through maturity.
- ▶ A late-date exercise spike can be investigated through regression and remaining-tail economics.

Takeaway. The statistics describe the fitted policy, not real-world exercise frequencies.

Martingale tests challenge the measure implementation

- ▶ The simulated domestic discount bond is compared with the USD curve target.
- ▶ Discounted FX-converted foreign zero-coupon value is compared with $S_0 P_f^*(0, T)$.
- ▶ Discounted FX-converted foreign bank account is compared with spot.

Takeaway. Z-scores assess whether sample differences are consistent with Monte Carlo error.

Diagnostics and MCP follow the core pricer

15 Aug
2026

- ▶ Local timestamps place bump/convergence diagnostics and the MCP server after the first pricing tests.
- ▶ The MCP exposes validation and single-deal pricing rather than a portfolio batch.
- ▶ Optional flags add common-random-number Greeks and convergence evidence.

Takeaway. The delivery interface grows only after a reusable `price()` function exists.

Rudimentary Greeks use full recalibration

- ▶ Central EURUSD spot delta scales forwards with spot to preserve F/S .
- ▶ USD and EUR parallel PV01 bump the respective OIS strips.
- ▶ Full- and half-bump values are compared with common random numbers.

Takeaway. The diagnostics explicitly omit gamma, vega, correlation, bucketed basis risk, and AAD.

Validation and the quantitative web page are added

16–17
Aug
2026

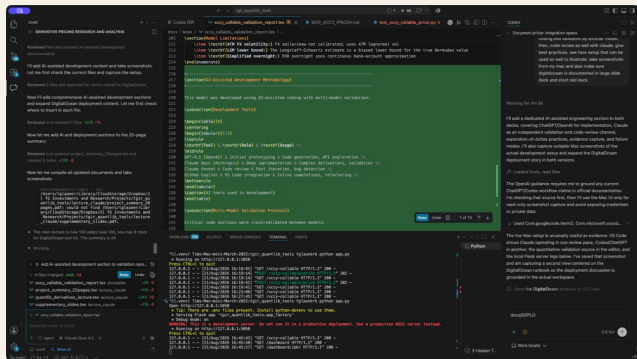
- ▶ A detailed validation report is born 16 August.
- ▶ The page context, template, and route tests follow on 17 August.
- ▶ The lab displays dynamics, calibration, value, exercise, martingales, limitations, and raw JSON.

Takeaway. Editable correlations trigger full recalibration and repricing from local market/deal files.

10 / Human-directed agent engineering

The pricer and the collaborative system that built it are co-equal achievements

The product was built inside a visible collaborative workspace



Takeaway. Tim, Codex, and Claude worked against the same inspectable Git repository, source, diffs, tests, specifications, and local server logs.

The human led the work; the agents performed most of the construction

- ▶ Tim set the economic questions, product scope, priorities, constraints, and acceptance standard, and repeatedly challenged the outputs.
- ▶ Codex produced most foundational specifications, implementation, integration code, documentation, and executable tests.
- ▶ Claude reviewed the Codex work and then built extensions around it for independent validation, regression testing, and future-change protection.

Takeaway. This was human-directed engineering by two active coding agents, not autocomplete and not a human manually transcribing generated snippets.

Codex was the primary specification and programming agent

- ▶ Codex translated conversational requirements into explicit deal semantics, model dynamics, calibration objectives, LSM recursion, interfaces, and acceptance criteria.
- ▶ It implemented the Python/QuantLib pricing core, JSON inputs, CLI, web page, editable correlation controls, MCP boundary, documentation, and deployment changes.
- ▶ It also wrote and ran unit, integration, calibration, convergence, Greek, bump-stability, and web smoke tests, then compiled and inspected the evidence.

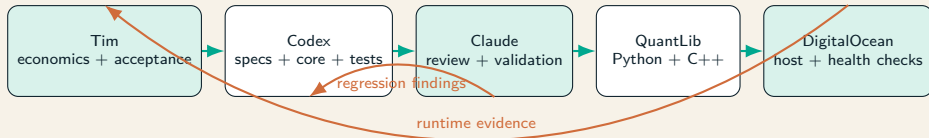
Takeaway. The specification, code, tests, and operating instructions were developed as one connected deliverable.

Claude turned review into a durable validation layer

- ▶ Claude first reviewed frozen Codex diffs findings-first, checking quantitative correctness, event ordering, numerical methods, APIs, security, and maintainability.
- ▶ More importantly, Claude added independent validation and regression extensions around the Codex core rather than stopping at prose comments.
- ▶ Those executable checks preserve benchmarks, reproduce accepted defects, and challenge later changes for drift signs, calibration, martingales, convergence, and bump behavior.

Takeaway. The second agent increased the system's future reliability by converting review conclusions into repeatable controls.

One cohesive framework connects people, agents, code, and hosting



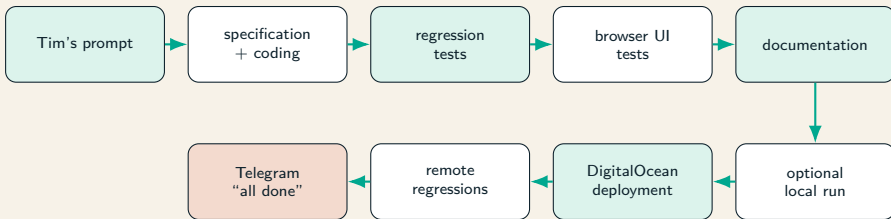
Takeaway. The Mac and Git repository are the shared engineering floor; a roughly USD 7/month DigitalOcean host makes this serious hobby continuously available.

Stack Overflow accelerated discovery but never decided correctness

- ▶ Community discussions helped locate QuantLib wrapper behavior, constructor changes, curve-horizon issues, multi-curve limitations, and known engine boundaries.
- ▶ Every useful suggestion was checked against the installed QuantLib version, upstream source or official documentation, and a minimal executable reproduction.
- ▶ Product-specific conclusions—especially the absence of a native callable XCCY composition—were established from current APIs and source, not an old answer alone.

Takeaway. Community knowledge shortened the search; version-pinned source inspection and tests converted it into evidence.

One prompt can drive the complete delivery loop



Takeaway. After the initiating prompt, the workflow can proceed without manual handoffs; the optional local run remains available when Tim wants to inspect it personally.

Evidence and authority remain explicit across both agents

- ▶ Freeze economic inputs and record commit, model/provider, prompt, permissions, commands, seeds, exit codes, reports, and known limitations.
- ▶ Keep secrets and unrestricted production data away from agent context; grant edits, shell, network, deployment, commit, and push authority only when needed.
- ▶ Require the human owner to triage findings, resolve disagreements, accept residual model risk, and authorize releases.

Takeaway. A persuasive response is not evidence; a reproducible specification, diff, benchmark, regression, and deployment check are.

The reusable achievement is an AI-native quantitative engineering method

- ▶ The complex three-factor callable XCCY pricer demonstrates that the method can handle coupled stochastic dynamics, calibration, Monte Carlo, and optimal stopping.
- ▶ The larger result is a repeatable framework for turning human intent into rigorous specifications, software, tests, independent challenges, and deployed evidence; disagreements remain visible and rerunnable.
- ▶ Neither cross-model agreement nor regression coverage creates formal independence, but the same division of labor can support native C++, stronger bounds, broader risk, and future QuantLib contributions.

Takeaway. The pricer is one major product; the low-cost, prompt-to-Telegram human-agent engineering capability is the compounding asset.

11 / 21 August 2026

Integration, validation, deployment, and the next model version

The XCCY body of work is committed as one model release

21 Aug
2026

- ▶ Commit f066cce adds 9,196 lines across 39 files.
- ▶ The message states: “Working XCCY callable 3-factor model with fixed vols.”
- ▶ The implementation was first deployed from the local commit and was subsequently pushed to GitHub `main`.

Takeaway. One commit contains a week of timestamped specification, implementation, diagnostics, UI, and validation work.

The MCP boundary supports guided single-deal pricing

- ▶ `xccy_validate_deal` returns readiness, issues, and concrete questions.
- ▶ `xccy_price_deal` prices one validated request and can include full audit output.
- ▶ `stdio` fits local agent integration; `loopback Streamable HTTP` fits controlled service integration.

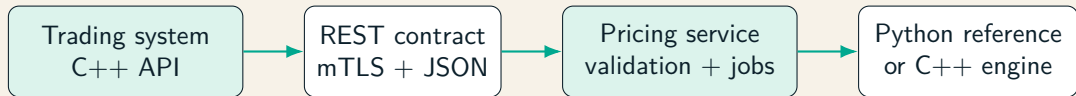
Takeaway. The server remains read-only and does not source missing market data automatically.

A C++ trading system should call a versioned REST contract

- ▶ The trading system owns a C++ REST client and sends normalized deal, market, model, numerics, and measure requests.
- ▶ Request IDs, idempotency keys, input hashes, status envelopes, and structured errors support replay.
- ▶ Synchronous validation is separated from potentially asynchronous pricing jobs.

Takeaway. Transport contracts should not expose QuantLib handles or Python implementation details.

Proposed production integration boundary



Takeaway. The wire contract can remain stable while the implementation migrates from Python to C++.

Validation evidence is broad but not tier-one approval

- ▶ The local release passed 46 application tests and 20 dedicated quantitative validation tests.
- ▶ Coverage includes bounds, martingales, monotonicity, calibration, reproducibility, convergence, edge correlations, and static benchmarks.
- ▶ The DigitalOcean Python 3.12 environment passed the same dedicated 20-test quantitative suite after deployment.

Takeaway. Formal organizationally independent validation, dual bounds, governed market data, and production controls remain outstanding.

The lower-bound gap is the central remaining numerical question

- ▶ An out-of-sample LSM policy avoids in-sample optimism but can still exercise suboptimally.
- ▶ More paths reduce sampling error without proving the policy is close to optimal.
- ▶ Representative trades need a trusted nested-MC benchmark or dual upper bound.

Takeaway. Grid, basis, seed ensemble, and regularization studies should accompany path convergence.

The next risk layer is market-structure sensitivity

- ▶ Add bucketed USD and EUR curve risk, cross-currency-basis risk, FX vega/smile risk, and correlation stresses.
- ▶ Separate forecast-basis from effective foreign-discount sensitivities.
- ▶ Stabilize bumps with common random numbers and compare full versus half bumps.

Takeaway. Document whether each scenario recalibrates rate and FX parameters.

The next model layer should be evidence driven

- ▶ Piecewise Hull-White rate volatility can improve diagonal fit but needs regularization and identifiability controls.
- ▶ FX smile may require local or stochastic volatility rather than more ATM buckets.
- ▶ Stochastic basis is a new factor and measure system, not a curve tweak.

Takeaway. C++ migration should follow profiling and interface stabilization, not precede them.

The end state is a platform for construction and challenge.

Tim directs → Codex specifies and builds

→ Claude reviews and extends validation

→ Mac proves → DigitalOcean hosts

A serious, rigorous hobby: QuantLib, Python, C++, community guidance, regression evidence, and a roughly USD 7/month host make the capability reusable beyond one pricer.